

Небольшой размер образа имеет много преимуществ: во-первых, загрузка небольшого образа из registry занимает гораздо меньше времени. Еще одно преимущество — безопасность: чем больше ваш образ, тем большую площадь для атаки он имеет.

Следующий учебник о "Building Small Containers" от Google — отличное видео, демонстрирующее важность оптимизации ваших Dockerfiles:

Building Small Containers



Прежде чем перейти к трюкам, показанным в видео, давайте начнем с уменьшения количества слоев образа. Что такое слой? Согласно [документации](#):

Чтобы создать свой собственный образ, вы создаете Dockerfile с простым синтаксисом для определения шагов, необходимых для создания образа и его запуска. Каждая инструкция в Dockerfile создает слой в образе. Когда вы изменяете Dockerfile и пересобираете образ, пересобираются только те слои, которые изменились. Это часть того, что делает образы такими легкими, маленькими и быстрыми по сравнению с другой виртуализацией.

Так что каждая команда, выполняемая к базовому образу, образует слой. Результирующий образ — это финальный слой, который объединяет изменения, содержащиеся во всех промежуточных слоях. Каждый слой потенциально добавляет что-то дополнительное к результирующему образу, так что может быть хорошей идеей минимизировать количество слоев.

Чтобы отслеживать улучшения, мы будем записывать размер образа после каждого нового Dockerfile. Начальная точка:

```
FROM ubuntu:22.04

WORKDIR /mydir

RUN apt-get update && apt-get install -y curl python3
RUN curl -L https://github.com/yt-dlp/yt-dlp/releases/latest/download/yt-dlp -o
/usr/local/bin/yt-dlp
RUN chmod a+x /usr/local/bin/yt-dlp

RUN useradd -m appuser
```

```
RUN chown appuser .
```

```
USER appuser
```

```
ENTRYPOINT ["/usr/local/bin/yt-dlp"]
```

Собранный образ имеет размер **155MB**

Как было сказано, каждая команда, выполняемая к базовому образу, образует слой. Команда здесь относится к одной директиве Dockerfile, такой как `RUN`. Мы могли бы теперь склеить все команды `RUN` вместе, чтобы уменьшить количество слоев, создаваемых при сборке образа:

```
FROM ubuntu:22.04
```

```
WORKDIR /mydir
```

```
RUN apt-get update && apt-get install -y curl python3 && \
    curl -L https://github.com/yt-dlp/yt-dlp/releases/latest/download/yt-dlp -o
    /usr/local/bin/yt-dlp && \
    chmod a+x /usr/local/bin/yt-dlp && \
    useradd -m appuser && \
    chown appuser .
```

```
USER appuser
```

```
ENTRYPOINT ["/usr/local/bin/yt-dlp"]
```

Размер образа **153MB**.

Разница не такая уж большая, образ с меньшим количеством слоев всего на 2 МБ меньше.

Как примечание, непосредственно не связанное с Docker: помните, что при необходимости можно привязать пакеты к версиям с помощью `curl=1.2.3` — это гарантирует, что если образ собирается позже, образ с большей вероятностью будет работать, так как версии точные. С другой стороны, пакеты будут старыми и иметь проблемы с безопасностью.

С помощью `docker image history` мы можем видеть, что наш единственный слой `RUN` добавляет 83.8 мегабайт к образу:

```
$ docker image history yt-dlp
```

IMAGE	CREATED	CREATED BY	SIZE
COMMENT			
a3f296f27a17	3 minutes ago	ENTRYPOINT ["/usr/local/bin/yt-dlp"]	0B
buildkit.dockerfile.v0			
<missing>	3 minutes ago	USER appuser	0B
buildkit.dockerfile.v0			
<missing>	3 minutes ago	RUN /bin/sh -c apt-get update && apt-get ins...	83.8MB
buildkit.dockerfile.v0			
...			

Следующий шаг — удалить все, что не нужно в финальном образе. Нам больше не нужны списки источников apt, так что мы можем приклеить следующую строку к нашему единственному `RUN`

```
.. && \  
rm -rf /var/lib/apt/lists/*
```

Теперь, после сборки, размер слоя **108 мегабайт**. Мы можем оптимизировать еще дальше, удалив `curl` и все зависимости, которые он установил. Это делается расширением команды следующим образом:

```
.. && \  
apt-get purge -y --auto-remove curl && \  
rm -rf /var/lib/apt/lists/*
```

Это снижает нас до **104 МБ**.

Упражнение 3.6

:::info Упражнение 3.6

Вернитесь теперь к нашему [frontend](#) и [backend](#) Dockerfile.

Зафиксируйте размеры обоих образов на данный момент, как это было сделано в материале. Оптимизируйте Dockerfiles обоих приложений frontend и backend, объединив команды RUN и удалив бесполезные части.

После ваших улучшений зафиксируйте размеры образов снова.

:::

Вариант Alpine Linux

Наш базовый образ Ubuntu добавляет больше всего мегабайт к нашему образу. [Alpine Linux](#) предоставляет популярную альтернативную базу в https://hub.docker.com/_/alpine/, которая составляет около 8 мегабайт. Она основана на альтернативной реализации glibc [musl](#) и бинарниках [busybox](#), так что не все программы работают хорошо (или вообще) с ней, но наш контейнер должен работать нормально. Мы создадим следующий файл `Dockerfile.alpine` :

```
FROM alpine:3.19  
  
WORKDIR /mydir  
  
RUN apk add --no-cache curl python3 ca-certificates && \  
    curl -L https://github.com/yt-dlp/yt-dlp/releases/latest/download/yt-dlp -o  
    /usr/local/bin/yt-dlp && \  
    chmod a+x /usr/local/bin/yt-dlp && \  
    adduser -D appuser && \  
    chown appuser . && \  
    apk del curl  
  
USER appuser
```

```
ENTRYPOINT ["/usr/local/bin/yt-dlp"]
```

Размер результирующего образа **57.6MB**

Примечания:

- Менеджер пакетов — `apk`, и он может работать без предварительной загрузки источников (кэшей) с помощью `--no-cache`.
- Для создания пользователя отсутствует команда `useradd`, но можно использовать `adduser`.
- Большинство имен пакетов одинаковы — есть хороший браузер пакетов на <https://pkgs.alpinelinux.org/packages>.

Мы собираем этот файл с `:alpine-3.19` как тег:

```
$ docker build -t yt-dlp:alpine-3.19 -f Dockerfile.alpine .
```

Кажется, работает нормально:

```
$ docker run -v "$(pwd):/mydir" yt-dlp:alpine-3.19 https://www.youtube.com/watch?v=bNw2i-mRT4I
```

Из истории мы видим, что размер нашего единственного слоя `RUN` составляет 49.8MB

```
$ docker image history yt-dlp:alpine-3.19

...
<missing>      6 minutes ago   RUN /bin/sh -c apk add --no-cache curl pytho...
49.8MB        buildkit.dockerfile.v0
...
<missing>      7 weeks ago     /bin/sh -c #(nop) ADD file:d0764a717d1e9d0af...
7.73MB
```

Итак, в общей сложности, наш вариант Alpine составляет около 57.6 мегабайт, значительно меньше, чем наш образ на основе Ubuntu.

Образ с предустановленной средой

Как видно, `yt-dlp` требует Python для работы. Установка Python в образ на основе Ubuntu или Alpine очень проста, это можно сделать одной командой. В общем случае установка среды, необходимой для сборки и запуска программы внутри контейнера, может быть довольно обременительной.

К счастью, есть предустановленные образы для многих языков программирования, доступные на DockerHub, и вместо полагания на "ручные" шаги установки в Dockerfile, довольно часто хорошей идеей является использование предустановленного образа.

Давайте используем тот, который сделан для [Python](#), для запуска `yt-dlp`:

```
# we are using a new base image
FROM python:3.12-alpine
```

```
WORKDIR /mydir
```

```
# no need to install python3 anymore
```

```
RUN apk add --no-cache curl ca-certificates && \
    curl -L https://github.com/yt-dlp/yt-dlp/releases/latest/download/yt-dlp -o
    /usr/local/bin/yt-dlp && \
    chmod a+x /usr/local/bin/yt-dlp && \
    adduser -D appuser && \
    chown appuser . && \
    apk del curl
```

```
USER appuser
```

```
ENTRYPOINT ["/usr/local/bin/yt-dlp"]
```

Есть много вариантов для образов Python, мы выбрали *python:3.12-alpine*, который имеет Python версии 3.12 и основан на Alpine Linux.

Размер результирующего образа **59.5MB**, так что он немного больше предыдущего, где мы устанавливали Python сами.

Еще в части 1 мы опубликовали Ubuntu версию yt-dlp с тегом *latest*.

Мы можем публиковать любые варианты, которые хотим, не перезаписывая другие, публикуя их с описывающим тегом:

```
$ docker image tag yt-dlp:alpine-3.19 <username>/yt-dlp:alpine-3.19
$ docker image push <username>/yt-dlp:alpine-3.19
$ docker image tag yt-dlp:python-alpine <username>/yt-dlp:python-alpine
$ docker image push <username>/yt-dlp:python-alpine
```

Или если мы больше не хотим держать Ubuntu версию, мы можем заменить ее, пушिंग образ на основе Alpine как latest. Кто-то может зависеть от того, что образ является Ubuntu.

```
$ docker image tag yt-dlp:python-alpine <username>/yt-dlp
$ docker image push <username>/yt-dlp
```

Важно помнить, что если не указано, тег `:latest` просто относится к самому последнему образу, который был собран и запущен, который потенциально может содержать любые обновления или изменения.

Упражнение 3.7

:::info Упражнение 3.7

Как вы, возможно, догадались, вы должны теперь вернуться к frontend и backend из предыдущего упражнения.

Измените базовый образ в FROM на что-то более подходящее. Чтобы избежать лишних хлопот, хорошей идеей является использование предустановленного образа как для [Node.js](#), так и для [Golang](#). Оба должны иметь по крайней мере варианты Alpine, готовые в DockerHub.

Обратите внимание, что frontend требует Node.js версии 16 для работы, так что вам нужно искать немного более старый образ.

Убедитесь, что приложение по-прежнему работает после изменений.

Зафиксируйте размер до и после ваших изменений.

...

Многоэтапные сборки

Многоэтапные сборки полезны, когда вам нужны некоторые инструменты только для сборки, но не для выполнения образа (то есть для CMD или ENTRYPOINT). Это простой способ уменьшить размер в некоторых случаях.

Давайте создадим веб-сайт с [Jekyll](#), соберем сайт для production и обслужим статические файлы с помощью Nginx. Начнем с создания рецепта для Jekyll, чтобы собрать сайт.

```
FROM ruby:3

WORKDIR /usr/app

RUN gem install jekyll
RUN jekyll new .
RUN jekyll build
```

Это создает новое Jekyll приложение и собирает его. Мы собираемся использовать Nginx для обслуживания страницы сайта, но вы можете протестировать, как работает сайт, если добавите следующую директиву:

```
CMD bundle exec jekyll serve --host 0.0.0.0
```

Мы могли бы начать думать об оптимизациях на этом этапе, но вместо этого мы добавим новый FROM для Nginx, это будет результирующий образ. Затем мы скопируем собранные статические файлы из Ruby образа в наш Nginx образ:

```
# the first stage needs to be given a name
FROM ruby:3 AS build-stage
WORKDIR /usr/app

RUN gem install jekyll
RUN jekyll new .
RUN jekyll build

# we will now add a new stage
FROM nginx:1.19-alpine

COPY --from=build-stage /usr/app/_site/ /usr/share/nginx/html
```

Теперь Docker копирует содержимое из первого образа `/usr/app/_site/` в `/usr/share/nginx/html`. Обратите внимание на именование из Ruby в *build-stage*. Мы также могли

бы использовать внешний образ как этап, например, `--from=python:3.12` .

Давайте соберем и проверим разницу в размерах:

```
$ docker build . -t jekyll
$ docker image ls
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
jekyll        nginx     9e2f597ad99e   8 seconds ago  21.3MB
jekyll        ruby     5dae3d9f8dfb   26 minutes ago 1.05GB
```

Как видите, хотя нашему образу Jekyll требовался Ruby во время этапа сборки, он значительно меньше, так как в результирующем образе есть только Nginx и статические файлы. `docker run -it -p 8080:80 jekyll:nginx` также работает как ожидается.

Часто лучшим выбором является использование FROM **scratch** образа, так как в нем нет ничего, что мы не добавили явно, что делает его самым безопасным вариантом со временем.

Упражнения 3.8 - 3.10

:::info Упражнение 3.8: Многоэтапный frontend

Сделайте теперь многоэтапную сборку для примера [frontend](#).

Хотя многоэтапные сборки разработаны в основном для бинарников, мы можем использовать преимущества с нашим frontend проектом, так как наличие исходного кода с финальными ресурсами мало имеет смысла. Соберите его с инструкциями в README, и собранные ресурсы должны быть в папке `build` .

Вы все еще можете использовать `serve` для обслуживания статических файлов или попробовать что-то другое.

:::

:::info Упражнение 3.9: Многоэтапный backend

Давайте сделаем многоэтапную сборку для [backend](#) проекта, так как мы зашли так далеко с приложением.

Проект написан на Golang, и сборка бинарника, который работает в контейнере, хотя и проста, не совсем тривиальна. Используйте доступные ресурсы (Google, примеры проектов), чтобы собрать бинарник и запустить его внутри контейнера, который использует `FROM scratch` .

Для успешного выполнения упражнения образ должен быть меньше **25MB**.

:::

:::info Упражнение 3.10

Сделайте все или большинство оптимизаций от безопасности до размера для **одного** другого Dockerfile, к которому у вас есть доступ, в вашем собственном проекте или, например, для тех, которые использовались в предыдущих "автономных" упражнениях.

Пожалуйста, задокументируйте Dockerfiles как до, так и после.

:::